让我佩服地五体投地的S.C. Park

文中好像没有交代局部极值点怎么找寻。

ELSEVIER

Computer-Aided Design 37 (2005) 451–460

COMPUTER-AIDED DESIGN

www.elsevier.com/locate/cad

Hollowing objects with uniform wall thickness

Sang C. Park*

Department of Industrial Information and Systems Engineering, Ajou University, San 5, Woncheon-dong, Yeongtong-gu, Suwon 443-749, South Korea

Received 29 June 2004; received in revised form 6 August 2004; accepted 9 August 2004

Abstract

In rapid prototyping, we can significantly reduce the building time and expensive RP material consumption by building a hollowed prototype instead of a solid model. Presented in the paper is a new method to hollow out solid objects with uniform wall thickness to speed up the part building processes in rapid prototyping systems. To achieve uniform wall thickness, it is necessary to generate internal contours by slicing the offset model of an STL model. Since computing an offset model of an STL model involves many difficulties, this paper proposes a new algorithm that computes internal contours without computing the offset model. The proposed algorithm is efficient and relatively easy to implement, because it employs well-known 2D geometric algorithms, such as planar curve offsetting and tracing innermost curves. Various examples have been tested with implementation of the algorithm, and some examples are presented for illustration.
© 2004 Elsevier Ltd. All rights reserved.

Keywords: Rapid prototyping; Hollowing out a solid; Uniform wall thickness; Circle swept volume; 2D curve offsetting

1. Introduction

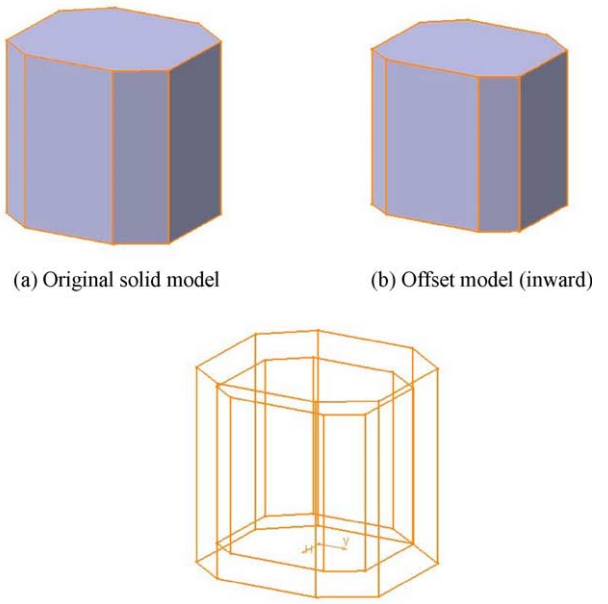
Rapid prototyping (RP) produces a physical prototype from a CAD model by building a part layer by layer in shorter time than more traditional prototyping methods such as milling, turning, etc. Unlike the traditional material removal processes, most rapid prototyping processes have to build a solid volume by gradually adding or solidifying materials layer by layer [1,2]. The process involves slicing the CAD model perpendicular to the building direction and manufacturing each layer sequentially from bottom to top. Tracing of the cross-sectional solid area in each layer is the most time consuming process, and it is possible to accelerate the process by reducing the cross-sectional area to be traced. One method that can be employed to reduce the cross-sectional area is hollowing out the model, as shown in Fig. 1. Since the hollowing operation reduces the area to be traced, depositing or solidifying the material on lesser area will not only reduce the build time, but also reduce material costs due to expensive RP build material [11,13].

RP applications [2] are expanding rapidly. Some examples of these applications are the making of molds for various casting operations (investment casting, die casting and sand casting), making parts for use in the assembly of products, and making parts required for medical applications. For many of these applications, it is very important to maintain uniform wall thickness in hollowing out a RP part [11,12]. For example, a RP part can serve as a core enclosed by a ceramic shell. The core will then be burnt to leave a cavity. A core with non-constant thickness can result in non-even shrinkage, which may break the ceramic shell during solidification [12].

For hollowing a solid model, several methods have been developed, and they can be classified into four categories: (1) spatial enumeration techniques such as the octree and the voxel model; (2) constructive solid geometry (CSG) offsetting methods; (3) curve offsetting methods; and (4) surface offsetting methods. Lam et al. [14] obtained a thin-shell solid model by using a sub-boundary octree, which was located in the interior of the original solid. Chiu and Tan [10] employed voxel elements to perform the hollowing. To avoid the difficulty of three-dimensional Boolean operations, they performed one-dimensional Boolean operations between the ray representations of the model and voxel elements. Alexander and Dutta [15] also used voxel elements to calculate the uniform wall thickness of a RP model. Since enumeration methods use octree or voxel

* Tel.: +82 2 3664 4700; fax: +82 31219 1610.
E-mail address: mar123@orgio.net.

0010-4485//$ - see front matter © 2004 Elsevier Ltd. All rights reserved.
doi:10.1016/j.cad.2004.08.001



(a) Original solid model (b) Offset model (inward)

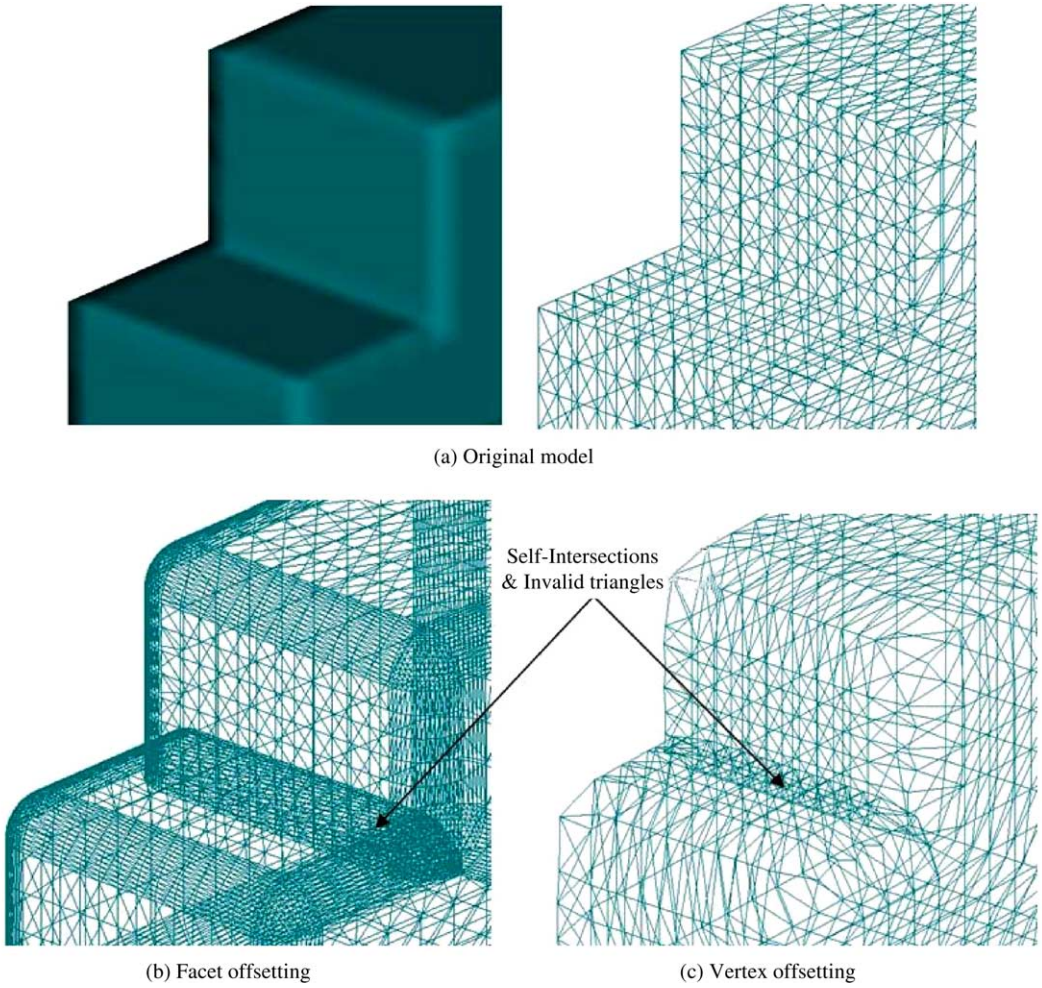
(c) Hollowed model = original model – offset model

Fig. 1. Hollowing out a solid model.

elements to hollow out a solid model, they can cause an internal staircase effect, which is one of the inherent attributes of octree and voxel elements.

Li et al. [12] proposed a method to obtain a thin-shell solid by subtracting the original solid from its offset counterpart. A CSG model is very easy to offset, because it is defined by a combination of simple primitives such as cones, boxes, cylinders and spheres. Their method is only applicable to CSG solids, however, because it is a non-trivial problem to offset parts in a boundary representation or other faceted approximations such as STL models.

Ganesan and Fadel [16] used a 2D curve offsetting approach to hollow out a solid model. By slicing the original part, external cross-sectional curves could be obtained, and they performed 2D curve offsetting to find internal cross-sectional curves. Since it employs 2D curve offsetting instead of 3D surface offsetting, this approach is relatively easy to implement. However, it cannot guarantee uniform wall thickness, as indicated by Koc and Lee [11], because the wall thickness depends on the surface normals, which are ignored in this approach.



(a) Original model

(b) Facet offsetting (c) Vertex offsetting

Fig. 2. Facet offsetting and vertex offsetting.

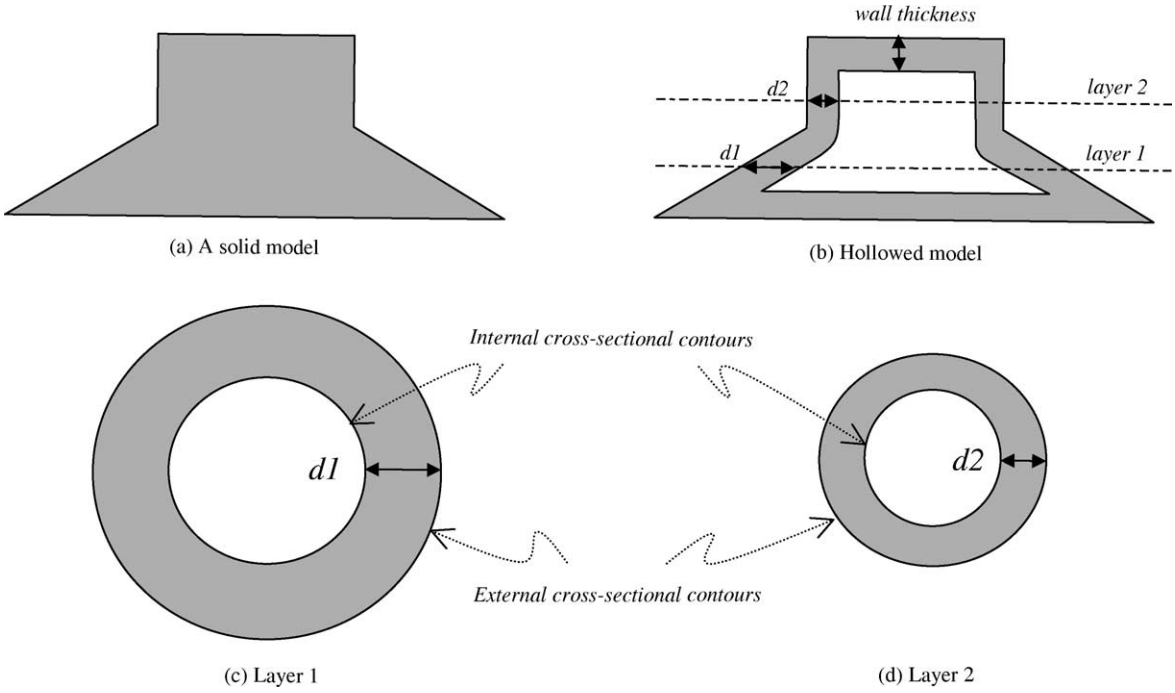


Fig. 3. Offsetting cross-sectional contours and its problem.

Koc and Lee [11] proposed a non-uniform offsetting method based on an averaged surface normals method to hollow out a solid model. Practically, computing the correct offset model (facet offsetting) of a STL model is not an easy task [11,17,18] because of the difficulty of regulation, which involves finding all self-intersections, removing invalid triangles, and filling gaps after raw offsetting. As shown in Fig. 2(a), the raw offset model has many more triangles than the original model, as well as many self-intersections and invalid triangles. In an effort to resolve this problem, Koc and Lee employed a vertex offsetting approach (Fig. 2(b)) based on the averaged surface normals method. Since the proposed method performs offsetting by moving each vertex with the estimated normal direction, the number of triangles does not increase after raw offsetting. While this method is very attractive and yields reasonable results for many cases, there are several potential problems: (1) It estimates a vertex normal by averaging normal vectors of the facets that are connected to the vertex, and as such the accuracy of the estimated vertex normal is not guaranteed, especially for sharp vertices; (2) Although the vertex offsetting approach will produce a less complicated raw offset model, the raw offset model will still include many self-intersections and invalid triangles, which should be regulated.

The approach utilized in this paper falls into the third category, curve offsetting methods, but it is significantly different from a simple 2D curve offsetting method. As noted earlier, it is not possible to achieve uniform wall thickness by simply offsetting 2D cross-sectional contours. Fig. 3(a) shows a very simple solid model, and Fig. 3(b) shows its hollowed model with uniform wall thickness. Assuming that the cross-sectional contours are the same centered circles, Fig. 3(c) and (d) illustrates cross-sectional contours to achieve uniform wall thickness. The distance between external contours and internal contours varies according to the surface normals, which means it is not possible to compute internal contours by 2D offsetting external contours with the wall thickness. To overcome the weakness of the conventional curve offsetting methods, a new method is developed to generate internal contours that can achieve uniform wall thickness. Since the proposed method is based on 2D geometric algorithms including 2D curve offsetting, it is relatively simple and easy to implement compared to other methods employing 3D geometric algorithms.

The remainder of the paper is organized as follows. Section 2 presents the overall approach to the problem of hollowing out a solid model, and Section 3 proposes a detailed algorithm to compute internal contour curves. Illustrative examples are provided in Section 4. Finally, concluding remarks are presented in Section 5.

2. Approach to hollowing out a solid

The objective of the paper is to hollow out a solid part (STL model) with uniform wall thickness to speed up the rapid prototyping process. The input of the problem consists of an STL model, a given wall thickness, an acceptable variation in wall thickness, and the layer thickness for adding RP material. For each layer, we need to compute the cross-sectional part area, which is defined by a combination of external contours and internal contours. While external contours can be easily computed by slicing the original STL

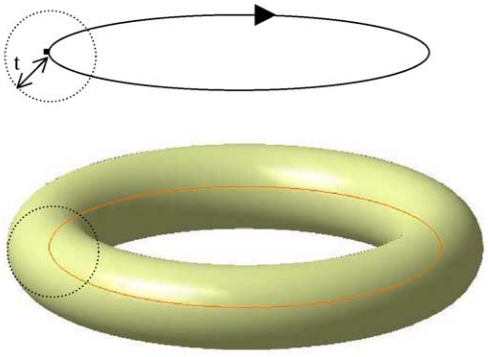
model, computing internal contours achieving uniform wall thickness is a non-trivial problem. Although, it is possible to compute internal contours by slicing the offset STL model, this paper proposes another method to avoid the difficulties in the STL model offsetting operation such as self-intersections and numerous invalid triangles.

The approach proposed here is to compute internal contours directly from a set of external contours. As shown in Fig. 4(a), we can obtain external contours by slicing the original solid model with uniformly distributed planes which are called '*E-Layers*'. If we assume that the offset surface of the solid model is available, it is simple to compute internal contours by slicing the offset surface. The key idea of the proposed approach is to generate the offset surface by using external contours for a given offset distance (t = wall thickness). As shown in Fig. 4(b), we can imagine a circle (radius = t) moving in space while its containing plane is kept orthogonal to the tangent direction of the external contour [8]. As a result of the circle movement, we can obtain a circle swept volume, as shown in Fig. 4(b). For each external curve, a circle swept volume can be defined, and it is possible to represent the offset surface by combining all circle swept volumes of external contours. As shown in Fig. 4(c), the sum of all circle swept volumes defines the inner offset surface as well as the outer offset surface. However, the offset surface defined by the sum of circle swept volumes is not perfectly accurate because of the discrete nature of the method. The accuracy depends on the gap distance between *E-Layers* containing external contours, and the gap distance should be determined by the required surface finish (acceptable variation in wall thickness) of the prototype to be built. It is possible to derive the gap distance from a given acceptable variation in wall thickness by using well known relations [19] between cusp height and path interval in NC machining. Fig. 5(a) shows how to determine the gap distance (path interval) based on the acceptable variation (cusp height) in wall thickness on a planar surface. For a give wall thickness t, the gap distance (g) between *E-Layers* is simply expressed in terms of the variation (v) in wall thickness as below:

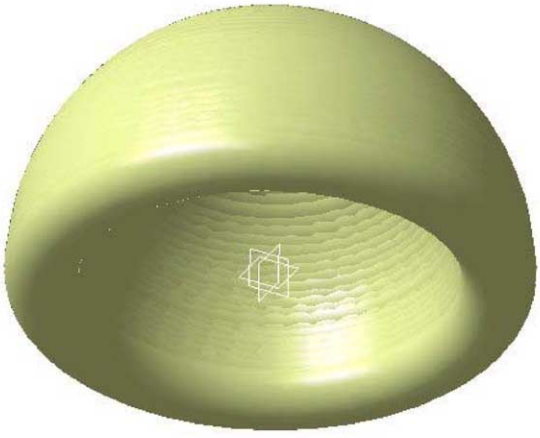
$$g = 2\sqrt{v(2t - v)}. \tag{2.1}$$



(a) A solid model & External Contours

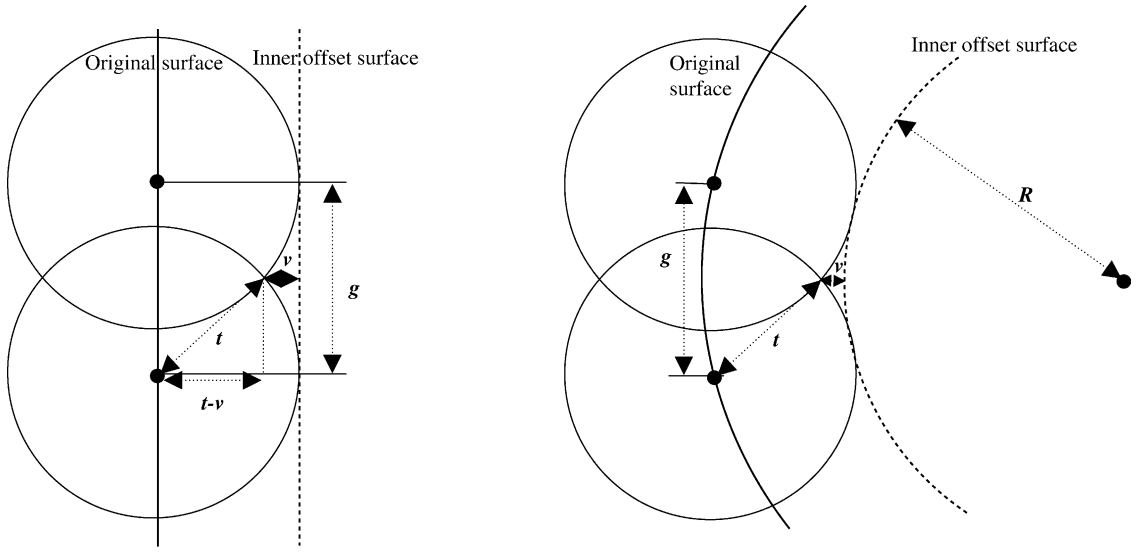


(b) An external contour & Circle swept volume



(c) Sum of swept volumes of all external contours

Fig. 4. Offset surface defined by the sum of circle swept volumes.



(a) Flat region (b) Convex region

t : wall thickness

v : acceptable variation in wall thickness

g : gap distance between layers containing external contours

Fig. 5. Variation in wall thickness and gap distance between E-Layers.

Now if the surface has a curvature along the direction normal to the part building direction, as shown in Fig. 5(b). The gap distance (g) between *E-Layers* can be computed from the following expression given in [19]

$$g = \frac{\sqrt{4(R+t)^2(R+v)^2 - [R^2 + 2Rt + (R+v)^2]^2}}{|R+v|}, \tag{2.2}$$

where R is the radius of normal curvature, v is the acceptable variation in wall thickness, and $t$is the wall thickness. In Eq. (2.2), both v and t are always positive; R is positive for a convex region and is negative for a concave region. The results presented above are mainly based on circular approximation of the surface, and they should be used with care.

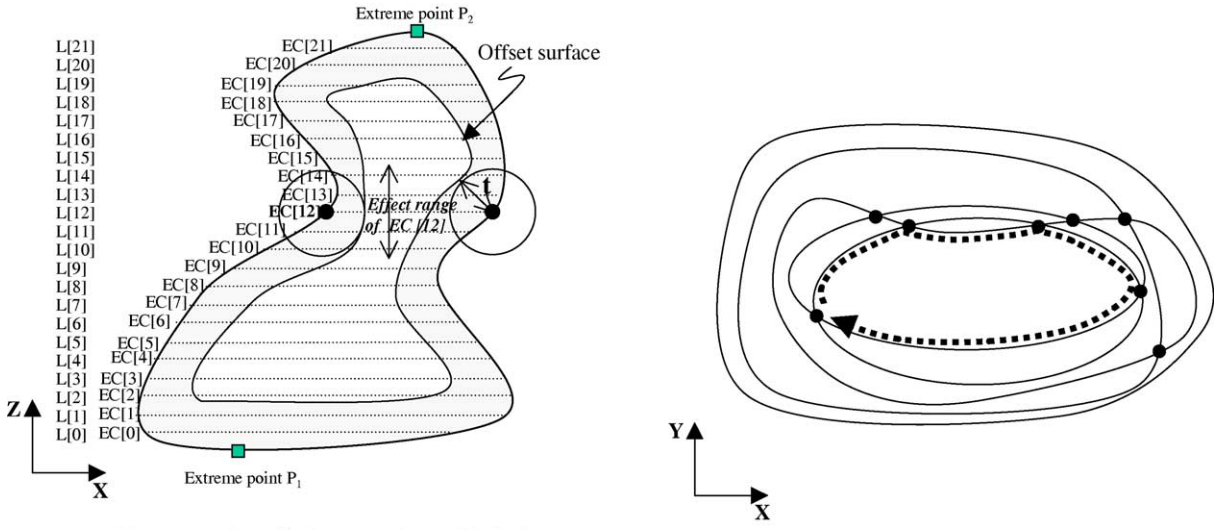
Once the inner offset surface becomes available, it is possible to compute internal contours by slicing the inner offset surface with slicing layers (*S-Layers*) which are determined by considering the layer thickness for adding RP material. The approach mentioned above can be roughly described as follows.

1. Determine gap distance of *E-layers* by considering the acceptable variation in wall thickness;
2. Compute external contours by slicing the original STL model with the *E-layers*;
3. Compute inner offset surface by combining all circle swept volumes of external contours;
4. Determine slicing layers (*S-Layers*) by considering the layer thickness for adding RP material;
5. Compute internal contours by slicing the inner offset surface with the *S-Layers*;
6. Compute external contours by slicing the original STL model with the *S-Layers*;
7. Define cross-sectional areas for each *S-Layer* by combining external and internal contours;

Conceptually, this approach can achieve uniform wall thickness with acceptable variation, because it computes internal contours by slicing the inner offset surface whose distance from the original model is the same as the wall thickness. However, in terms of implementation, there are two challenging problems: (1) if the shape of an external contour is not sufficiently simple, its circle swept volume can cause numerous self-intersections and invalid triangles; (2) the offset surface, defined by the sum of circle swept volumes, will include numerous interferences between those circle swept volumes, which may hamper the slicing process for computing correct internal contours. To avoid these difficulties, this paper develops an algorithm to generate internal contours without computing the offset surface or the circle swept volumes. A detailed description of the algorithm is provided in Section 3.

3. Computing internal cross-sectional contours

While the basic concept of the proposed algorithm is to compute internal contours by slicing the offset surface defined by the sum of circle swept volumes of external



(a) Effect range of the circle swept volume of EC[12] : L[10], L[11], L[12], L[13], L[14]

(b) Internal contour extraction

Fig. 6. Effect range of a circle swept volume.

contours, we do not want to compute the actual offset surface nor the circle swept volumes. Fig. 6(a) shows the side view of an example having 22 *E-Layers* (*L[i], $0 \le i \le 21$*) containing external contours (*EC[i]=an external contour on L[i], $0 \le i \le 21$*). Since an external contour lies on a plane (*E-Layer*), its circle swept volume has a limited affecting range on the offset surface. For example, the circle swept volume of *EC*[12], shown in Fig. 6(a), can potentially contribute to the limited portion of the offset surface including *L[10], , L[11], , L[12], L[13]* and *L[14]*. In other words, it is possible to compute the internal contour of *L[12]* by considering the circle swept volumes of *EC[10], EC[11], EC[12], EC[13]* and *EC[14]*. Instead of slicing the offset surface, we can slice the five circle swept volumes with the plane of *L[12]*. The slicing operation will give five sliced curves lying on the layer, *L[12]*. The next step is to find all intersections among those sliced curves. With the intersection points, we can extract the internal contour of *L[12]* by tracing the innermost curve, as shown in Fig. 6(b).

But the method mentioned above cannot be applied to layers where the internal contours do not exist (*L[0], L[1], L[20],* and *L[21]* in Fig. 6(a)). Let's assume that we want to compute the internal contours of *L[0]*. There are only three circle swept volumes of *EC[0], EC[1]* and *EC[2]* which possibly affect the computation, and they will lead to an internal contour curve which is wrong. Because of this problem, we introduce the concept of an '*extreme point*'. In this paper, we assume that the building direction is the *Z*-axis, and a vertex of an STL model is called an *extreme point* if its *Z*-value is locally extreme (minimum or maximum). Fig. 6(a) shows two extreme points (P_1, P_2) and they can be used for checking the validity of internal contours. For example, it is possible to know that the internal contour of *L[0]* is invalid because its distance from P_1 is smaller than the wall thickness. As a result, we know that *L[0]* does not have any internal contours, which means the whole area defined by the external contour should be filled with RP material.

To avoid the computation of the circle swept volumes, we need to develop a method to obtain a sliced curve directly from the information of an external contour and the slicing layer. Fig. 7(a) shows an external contour on an *E-Layer* (L_i) and its circle swept volume. Here, we want to slice the inner side of the circle swept volume, shown in Fig. 7(b), with a slicing layer (S_j). The distance (d) between L_i and $S_j$should be less than the wall thickness (t), which is the radius of the circle, otherwise there would be no intersections between the circle swept volume and the *S-Layer* S_j. Fig. 7(c) shows the computation procedure of the sliced curve, consisting of two steps: (1) translate the external contour with the distance d so that the external contour lies on the *S-Layer*; and (2) perform 2D offset inward with the distance $\sqrt{t^2 - d^2}$. As a result, we can obtain the sliced curve which is the intersection curve between the circle swept volume of EC_i and the *S-Layer* S_j. As shown in Fig. 7(d), we can obtain the sliced curve without computing the actual circle swept volume. The proposed algorithm can be described as follows (further explanation is also provided later).

3.1. Computing internal contours

// Input: *E-Layers* (L_i, $1 \le i \le n$) and each L_i contains at least one external contour curves
Wall thickness (***t***), *S-Layers* (S_i, $1 \le i \le m$) for storing internal contours
Extreme points (P_i, $1 \le i \le l$)
// Output: Slicing layers (S_i, $1 \le i \le m$) containing internal contours

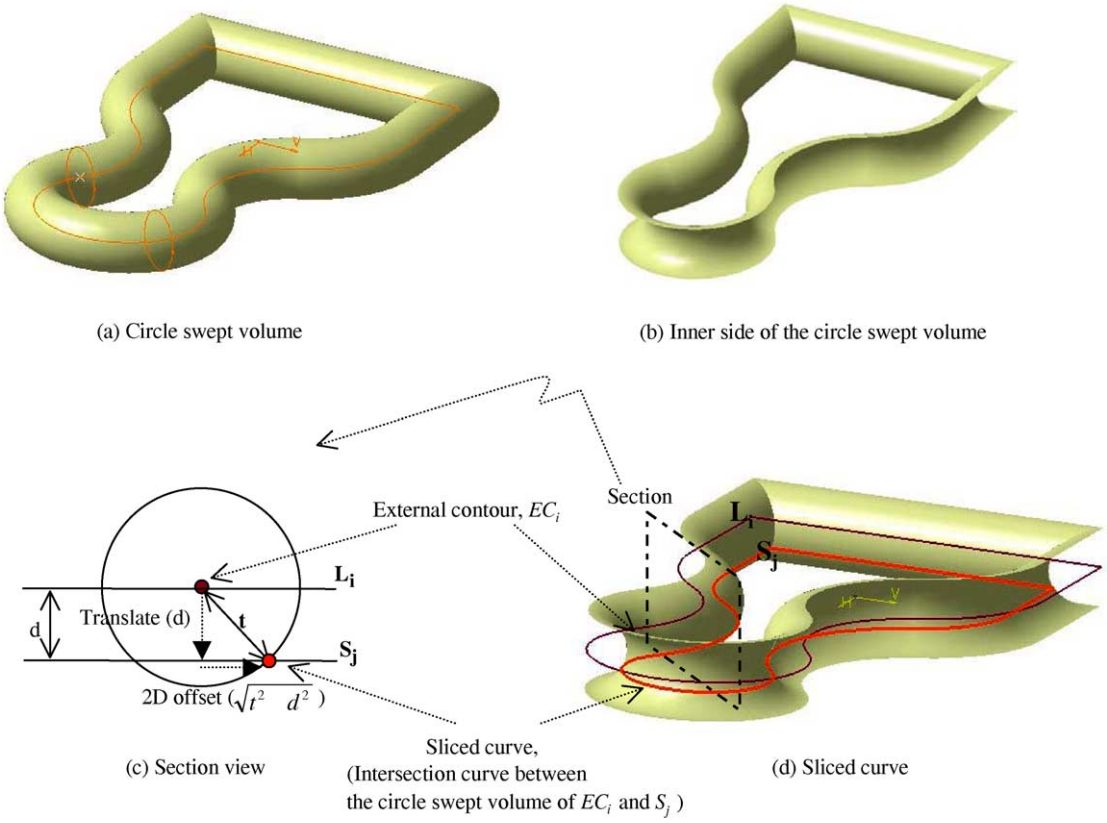
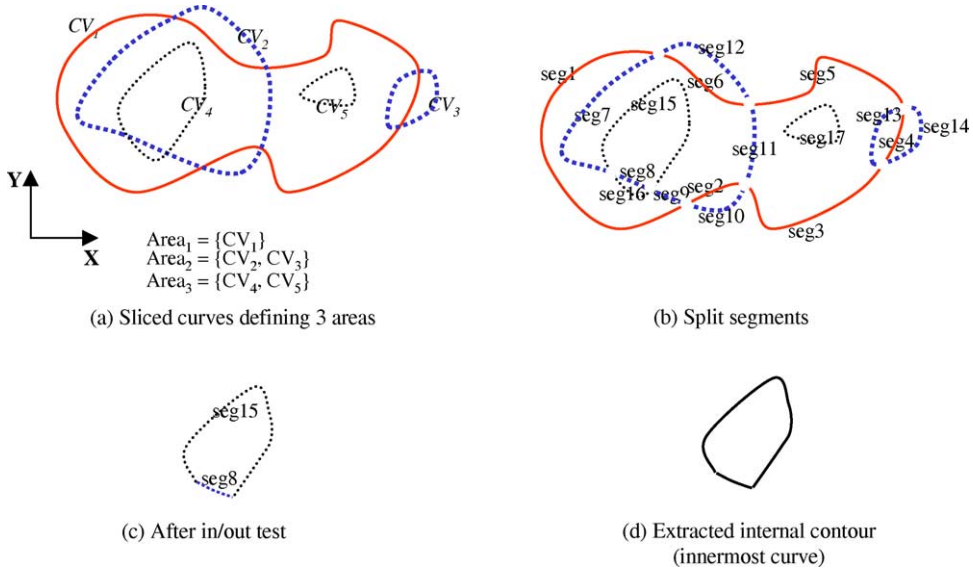


Fig. 7. Computing a sliced curve.

For ($i=1$; $i\leq m$; $i++$){ // For each slicing layer S_i
Step (1) Find a sequence of *E-layers* ($L_p \sim L_q$, $p<q$), whose distance from S_i is smaller than the wall thickness t;
Step (2) For ($j=p$; $j\leq q$; $j++$) { // For each *E-layer* ($L_p \sim L_q$)
Step (2-1) d=distance between S_i and L_j;
Step (2-2) Translate (external contour curves of L_j, d) so that they lie on S_i;
Step (2-3) Area$_j$=Perform 2D_offset (Translated external contours,$\sqrt{t^2-d^2}$);
Step (2-4) If (Area$_j$==NULL) { // If the 2D offsetting gives nothing.
S_i=NULL; // Internal contours do not exist
Go to Step 1; // Go to next slicing layer
}
}
Step (3) Find intersections between sliced curves (Area$_j$, $p\leq j\leq q$);
Step (4) S_i=Extract the innermost curves by tracing intersections;
Step (5) If the minimum distance between an innermost curve of S_i and extreme points (P_i, $1\leq i\leq l$) is smaller than $\boldsymbol{t}$, then delete the innermost curve from S_i;
}

In the algorithm described above, Step 1 is intuitive and Step 2–3 requires a 2D curve offsetting operation. One of the issues of a 2D offsetting operation [3–6] is the removal of invalid loops caused by self-intersections after raw offsetting. There are several algorithms that can handle invalid loops efficiently, and as such we can choose one of them by considering important factors of the specific application domain, such as the required robustness, simplicity and efficiency. Step 3 is an intersection finding problem among planar curves (point sequence curves) [7,9]. Park and Shin [7] proposed a polygonal chain intersection algorithm that finds intersections among point sequence curves by using the concept of monotone chains of a polygon. In terms of actual computation time, the algorithm is considerably better than conventional algorithms especially when the polygon consists of many small line segments. Step 4, extracts the internal contour by using intersection points found in the previous step. To explain the method extracting the innermost curves, Fig. 8(a) shows an example consisting of five sliced curves and three areas. Let's assume that those three areas ($Area_1$, $Area_2$, $Area_3$) are obtained from Step 2 and 3 for a certain *S-Layer*. We can split the sliced curves into curve segments by using intersection points, as shown in Fig. 8(b). We then need to test whether each curve segment is included in other areas to which it does not belong. For example, *seg7* (belonging to $Area_2$) cannot be a part of the innermost curve because it is in the outside of $Area_3$, while *seg15* (belonging to $Area_3$) can be a part of the innermost curve because it is included by both $Area_1$ and $Area_2$. In this way, we can simply construct the internal contour by connecting



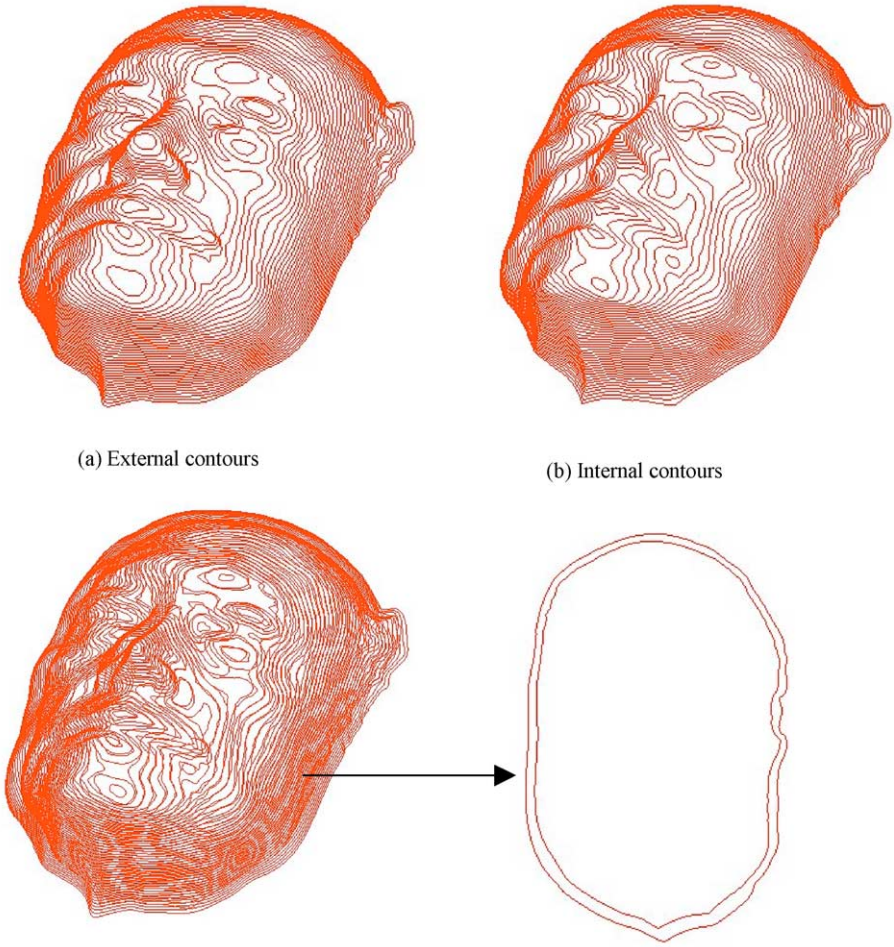
(a) Sliced curves defining 3 areas

(b) Split segments

(c) After in/out test

(d) Extracted internal contour (innermost curve)

Fig. 8. Extracting innermost curves.



(a) External contours

(b) Internal contours

(c) External & Internal contours
wall thickness: 7mm

(d) A pair of external & internal contours
on the same layer

Fig. 9. Example A: internal contours from external contours.

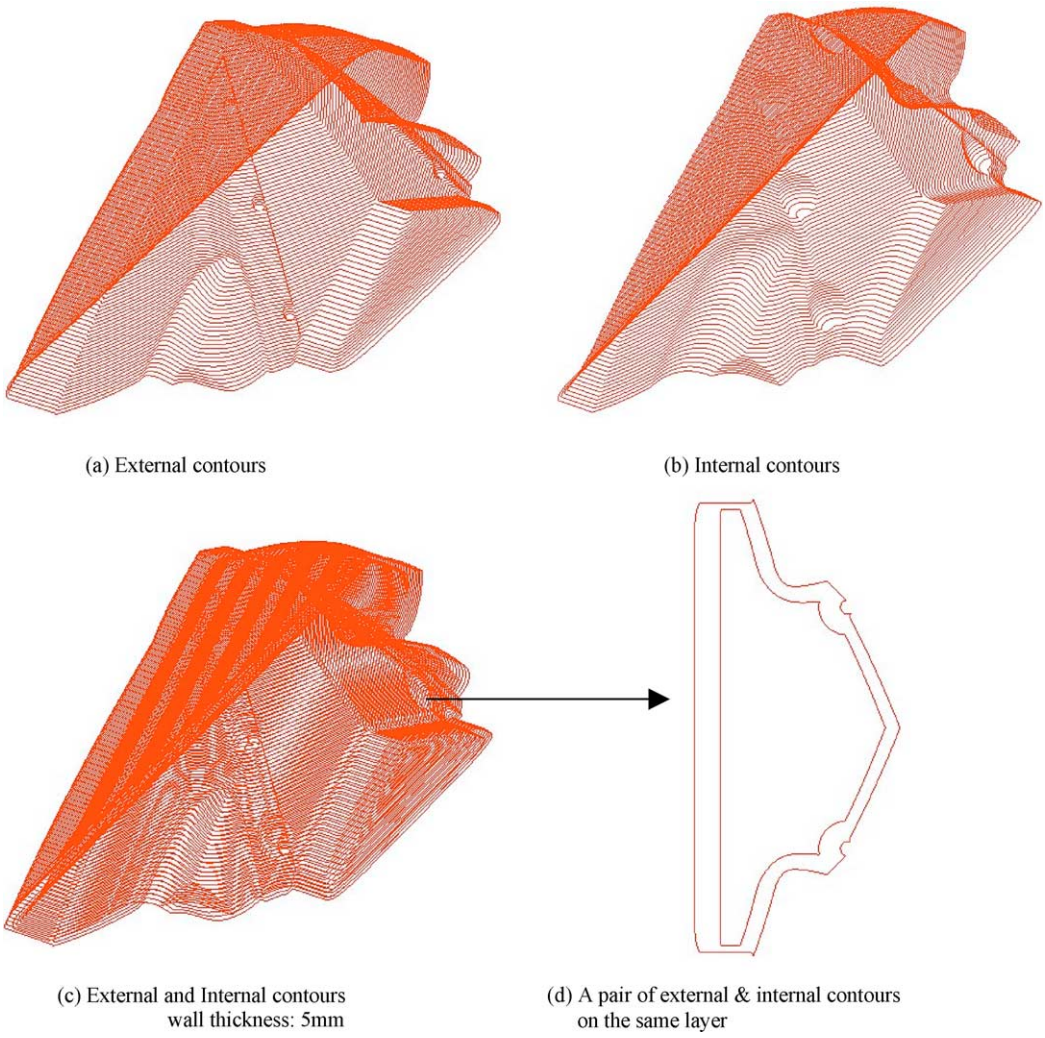


Fig. 10. Example B: internal contours from external contours.

curve segments included by all other sliced curves, as shown in Fig. 8(d).

The proposed algorithm computes internal contours directly from external contours without computing the offset surface or circle swept volumes. Although the algorithm does not actually slice the offset surface, the computed internal contours achieve uniform wall thickness because the algorithm implements the concept of slicing the offset surface with 2D geometric algorithms.

4. Illustrations

The proposed algorithm has been tested with various examples. Fig. 9(a) shows external contours generated by slicing a human face model (200 mm×250 mm×100 mm), and the computed internal contours are shown in Fig. 9(b). If the two groups of external and internal contours are rendered together, we can then see the difference (uniform wall thickness), as shown in Fig. 9(c). Since the internal contours are generated by slicing the offset surface, they achieve uniform wall thickness of the hollowed model. Fig. 9(d) shows a pair of external and internal contours, and we can see that the distance between them is not uniform. Fig. 10 shows another example which is a mechanical component.

The proposed algorithm was implemented in the C ++ language and test runs were made on a personal computer with a Pentium 4 processor with a 512 MB memory and a Windows XP operating system. For performance tests, the execution times were collected for the example shown in Fig. 9, and the results are shown in Table 1. The execution

Table 1
Execution times for the human face model (No. of *S-layers*: 100, wall thickness: 7 mm)

No. of E-layers	Gap distance between *E*-layers (mm)	Variation in wall thickness on a flat surface (mm)	Execution time
50	2	0.0717	38 s
100	1	0.0178	1 min 23 s
200	0.5	0.0044	3 min 12 s
400	0.25	0.0011	5 min 15 s

times in Table 1 may be affected by many practical factors (code optimization, data structure, CPU performance …), but they are good enough to show that the execution of the proposed approach can be completed within reasonable time.

5. Conclusions

This paper presents a procedure to hollow a solid model to accelerate the part building processes in rapid prototyping (RP) systems. For hollowing a solid model, it is necessary to generate internal contours that can achieve uniform wall thickness of a hollowed prototype. Although external contours can be easily obtained by slicing the solid model (STL), internal contours should be generated by slicing the offset surface for the uniform wall thickness. However, the offsetting of an STL model presents significant challenges. Practically, computing the exact offset model of an STL model is not an easy task because of the difficulty of regulation, which involves finding all self-intersections, filling gaps, and removing invalid triangles after raw offsetting. To avoid the difficulties in computing a correct offset model, this paper develops a new procedure to generate internal contours directly from the external contours. The underlying concept of the proposed algorithm is that the sum of circle swept volumes of external contours represents the offset model. While it is possible to compute an internal contour of a layer by slicing the circle swept volumes affecting the layer, it is not necessary to compute the actual circle swept volumes because we can generate the sliced curves with a simple combination of 2D geometric operations. Since the proposed algorithm is based on well-known 2D geometric algorithms, it is efficient and easy to implement. Using the proposed algorithm, we build a hollowed prototype instead of a solid part, which significantly reduces the building time and expensive RP material consumption.

References

[1] Jacobs P. Rapid prototyping and manufacturing fundamentals of stereolithography. Soc Manufact Eng 1992.
[2] Nyaluke AP, An D, Leep HR, Parsaei HR. Rapid prototyping: applications in academic institutions and Industry. Comput Ind. Eng 1995;29(1-4):345–9.
[3] Choi BK, Park SC. A pair-wise offset algorithm for 2D point-sequence curve. Comput Aided Des 1999;31(12):735–45.
[4] Held M. On the computational geometry of pocket machining. Berlin: Springer LNCS 500; 1991.
[5] Tiller W, Hanson EG. Offset of two-dimensional profiles. IEEE Comput Graphics Appl 1984;4:36–46.
[6] Chiang CS, Hoffman CM, Lynch RE. How to compute offsets without self-intersection. In SPIE Conf. Proc. curves and surfaces in computer vision and graphics II, Boston, MA 1991;76–87.
[7] Park SC, Shin H. Polygonal chain Intersection. Comput Graphics 2002;26:341–50.
[8] Park SC. Polygonal extrusion. Vis Comput 2003;19:38–49.
[9] Bentley JL, Ottmann TA. Algorithms for reporting and counting geometric intersections, IEEE Transact Comput 1979;30:123–9.
[10] Chiu WK, Tan ST. Using dexels to make hollow models for rapid prototyping. Comput Aided Des 1998;30(7):539–47.
[11] Koc B, Lee YS. Non-uniform offsetting and hollowing objects by using biarcs fitting for rapid prototyping processes. Comput Ind 2002; 47:1–23.
[12] Li CL, Yu KM, Lam TW. Implementation and evaluation of thin-shell rapid prototype. Comput Ind 1998;35:185–93.
[13] Yu KM, Li CL. Speeding up rapid prototyping by offset,. Proc Institut Mech Eng Part B: J Eng Manuf 1995;209:1–8.
[14] Lam TW, Yu KM, Cheung KM, Li CL. Octree reinforced thin-shell rapid prototyping. J Mater Process Technol 1997;63:784–7.
[15] Alexander P, Dutta D. Layered manufacturing of surfaces with open contours using localized wall thickening. Comput Aided Des 2000;32: 175–89.
[16] Ganesan M, Fadel GM. Hollowing rapid prototyping parts using offsetting techniques. Proceedings of the 5th international conference on rapid prototyping, Dayton, OH, June 1994.
[17] Xiuzhi Q, Brent S. A 3D surface offset method for STL-format models. Rapid Prototyping J 2003;9:133–41.
[18] Maekawa T. An overview of offset curves and surfaces. Computer-Aided Design 1999;31:165–73.
[19] Choi BK, Jerard RB. Sculptured surface machining. Dordetch: Kluwer; 1998.



Sang C. Park is an assistant professor in the Department of Industrial and Information Systems Engineering at Ajou University. Before joining Ajou, he worked for Daimler-Chrysler Corp. and CubickTek Co., developing commercial and in-house CAD/CAM/CAPP software systems. He received his BS, MS and PhD degrees from KAIST in 1994, 1996 and 2000, respectively, all in Industrial Engineering. His research interests include geometric algorithms in CAD/CAM, process planning, engineering knowledge management, and discrete event system simulation. He can be reached via email at mar123@orgio.net.